Modelling in Constraint Programming (CP)

Overview and Context

While Constraint Programming (CP) is fundamentally a declarative paradigm—meaning the user specifies *what* the problem is rather than *how* to solve it—the reality of computational limits makes **modelling** an essential skill. The way a problem is mathematically formulated dictates the size of the search space, which grows exponentially with the number of variables and the size of their domains. A poor model will leave the solver hopelessly lost in an intractable search tree, whereas an intelligent model provides the solver with the structural insights needed to aggressively prune dead-ends.

This topic explores the transition from a naive declarative specification to an optimized, solver-friendly architecture. The primary learning objective is to master advanced modelling techniques, such as the strategic introduction of auxiliary variables, the identification and elimination of symmetries, and the powerful concept of combining different problem representations through channeling constraints.

1. Formalization of CSP and COP

Explanation: To feed a problem into a CP solver, we must translate our real-world understanding into a rigorous mathematical structure. This structure must capture all unknowns, their possible values, and the rules governing them. Constraints can be expressed in various ways: extensional representations explicitly list all valid tuples, while intensional representations use logical or algebraic invariants. Because the order of constraints does not matter and they are non-directional, the solver uses them as a network of communicating rules to infer domain reductions. When we want to find the *best* solution rather than just *any* valid solution, we upgrade our problem to a Constraint Optimization Problem (COP) by introducing an objective function.

- **Constraint Satisfaction Problem (CSP):** Formally defined as a triple $< X, D, C >$:
  - X: A set of decision variables $\{X_1, \ldots, X_n\}$.
  - D: A set of domains $\{D_1, \ldots, D_n\}$, where D_i is the finite set of possible values for X_i.
  - C: A set of constraints $\{C_1, \ldots, C_m\}$. Each C_i is a relation over a subset of variables $\{X_j, \ldots, X_k\}$, formally represented as the allowed combinations of values: $C_i \subseteq D(X_j) \times \cdots \times D(X_k)$.

  A solution to a CSP is an assignment to the variables which satisfies all constraints **simultaneously**

- **Constraint Optimization Problem (COP):** A CSP enhanced with an objective. Formally $< X, D, C, f >$, where f is an objective variable

representing a criterion like minimum cost or maximum profit. The goal is to minimize f (or maximize $-f$).

- **Constraint Types:**
  - *Extensional:* Explicit tables of allowed tuples (e.g., $(X, Y) \in \{(0,0),(1,3)\}$). General, but inefficient for large domains.
  - *Intensional:* Algebraic or logical invariants (e.g., $X_1 + X_2 = X_3$). Compact and clear.
  - *Global:* High-level patterns capturing complex relations over many variables (e.g., `alldifferent([X1, ..., Xn])`), leading to superior domain pruning.

Examples & Applications

- **Sudoku:** Modelled with 81 variables (X_{11} to X_{99}) with domains $[1..9]$. The rules are enforced using global constraints: `alldifferent` on every row, column, and 3×3 sub-grid.
- **N-Queens**: Modelled with a variable for each row (or columnn) each in the $[0..N-1]$ integer domain, indicating the position of the queen in the row (or columnn)
- **Task Scheduling:** Variables $Start_i$ with domains $[0..D]$ represent task start times. Constraints ensure temporal logic (release dates $r_i \leq Start_i$) and prevent overlaps using non-linear logic: $(Start_i + p_i \leq Start_j) \vee (Start_j + p_j \leq Start_i)$, where p_i is the processing time.

2. Auxiliary Variables and Implied Constraints

Explanation: Often, the most intuitive set of variables (the "Naive Model") makes it mathematically cumbersome to express the constraints of the problem. This leads to high-complexity constraint formulas that offer weak propagation. To fix this, we introduce **Auxiliary Variables**—new variables that are not strictly necessary to output the final solution, but act as intermediate mathematical steps. By tying the original variables to these auxiliary variables, we can often re-write complex constraints into much simpler, lower-complexity forms, or take advantage of Global Constraints.

- **Auxiliary Variables:** New variables introduced to make constraints easier to express or to achieve stronger domain reductions.
- **Implied (Redundant) Constraints:** Constraints that are logically implied by the original problem definitions (semantically redundant) but cannot be deduced by the solver immediately. Adding them explicitly helps the solver reduce the search space drastically (computationally significant).

Examples & Applications

The Golomb Ruler Case Study: Place m marks on a ruler to minimize its length so that all distances between marks are unique. * *Naive Model:* Variables

X_i for mark positions. The constraint $|X_{i1} - X_{j1}| \neq |X_{i2} - X_{j2}|$ requires comparing every pair of pairs, resulting in computationally heavy **quartic** $\mathcal{O}(m^4)$ constraints. * *Better Model:* Introduce auxiliary variables D_{ij} representing the distance between mark i and mark j. We link them: $D_{ij} = |X_i - X_j|$. Now, we simply state `alldifferent([D12, D13, ..., D(m-1)m])`. This reduces the complexity to **quadratic** $\mathcal{O}(m^2)$ and leverages a highly optimized global constraint.

3. The Problem of Symmetry

Explanation: Symmetry occurs when a problem has multiple structurally identical states. If a solver encounters a dead-end, it will backtrack and may unknowingly explore a symmetrically equivalent dead-end, wasting massive amounts of time (a phenomenon known as "thrashing"). Symmetries can exist in the variables or the values themselves. The standard defense mechanism is to add **Symmetry Breaking Constraints**. These are artificial rules imposed by the modeller to force the solver to consider only one "canonical" version of a solution, effectively pruning all its symmetric twins from the search tree.

- **Variable Symmetry:** Occurs when there is a permutation π of the variable indices that transforms any valid assignment into another valid assignment (e.g., permuting identical machines in a factory). If there are m symmetric variables, there are $m!$ symmetries.
- **Value Symmetry:** Occurs when there is a permutation π of the domain values that transforms any valid assignment into another (e.g., swapping the colors "Red" and "Blue" across a map).
- **Symmetry Breaking Constraints:** Artificial limits (usually orderings) added to the model. *Crucial rule:* You must ensure that at least one solution from each set of symmetrically equivalent solutions survives.

Examples & Applications

Symmetry in the Golomb Ruler: A ruler $\{0, 1, 4, 6\}$ is physically the same as its reversed counterpart $\{0, 2, 5, 6\}$. This is a value symmetry ($0 \to 0, 1 \to 2$, etc.). To break this, we impose: 1. **Variable Ordering:** $X_1 < X_2 < \cdots < X_m$ (forces marks to be sorted). 2. **Distance Asymmetry:** $D_{12} < D_{(m-1)m}$ (forces the first distance gap to be smaller than the last, breaking the reverse flip symmetry).

4. Dual Viewpoints and Channeling

Explanation: The hardest part of modelling is that a single problem can often be viewed from entirely different mathematical perspectives, each with its own pros and cons. One viewpoint might allow the use of powerful global constraints, while another viewpoint makes it incredibly easy to break complex geometric symmetries. When it is too difficult to choose the "best" model, modern CP encourages us to use **Combined Models**. We keep both sets of variables and

connect them using **Channeling Constraints**. This forces the two models to synchronize during the search, allowing the solver to benefit from the strengths of both representations simultaneously.

- **Dual Viewpoint:** Representing problem P with completely different variables and domains. The search space size and propagation strength will vary drastically between viewpoints.
- **Lexicographic Ordering ($lex \leq$):** A constraint that requires sequence A to be alphabetically/lexicographically smaller than or equal to sequence B. Excellent for breaking symmetries in matrices.
- **Channeling Constraints:** Logical bridges linking the variables of Model A to the variables of Model B to maintain consistency.

Examples & Applications

Optimizing the N-Queens Problem: * *Integer Model (Alldiff):* Variables $X_i \in [1..n]$ represent the column of the queen in row i. * *Math translation:* Diagonal attacks $|X_i - X_j| \neq |i - j|$ can be algebraically rewritten as $X_i - i \neq X_j - j$ and $X_i + i \neq X_j + j$. * *Result:* We can use global constraints: `alldifferent([X_1 - 1, X_2 - 2, ...])`. Excellent for propagation, but terrible for breaking the 8 geometric symmetries (rotations/flips) of a chessboard. * *Boolean Model:* An $n \times n$ matrix of binary variables $B_{ij} \in \{0, 1\}$. It cannot use `alldifferent`, but geometric symmetries are easily broken by forcing the matrix to be $lex \leq$ than its 7 rotated/flipped permutations. * *The Combined Solution:* We use both X_i and B_{ij}. We link them with the Channeling Constraint: $X_i = j \iff B_{ij} = 1$. The solver uses X_i for fast `alldifferent` propagation and B_{ij} to cut off geometric symmetries via $lex \leq$.

Synthesis and Conclusions

Effective Constraint Programming extends far beyond defining a problem; it requires architecting a search space. We began by establishing the formal mathematics of a CSP, recognizing that an exponential search tree requires aggressive pruning. By observing the Golomb Ruler, we learned that introducing auxiliary variables and implied constraints can reduce computational complexity from quartic to quadratic. We tackled the devastating inefficiency of symmetry by artificially ordering our variables, preventing redundant exploration. Ultimately, the exploration of Dual Viewpoints in the N-Queens problem revealed the pinnacle of CP modelling: instead of compromising between propagation strength and symmetry breaking, we can construct combined models linked by channeling constraints, achieving peak solver efficiency.

Essential Glossary

- **Constraint Satisfaction Problem (CSP):** A mathematically formalized problem defined by a triple $< X, D, C >$ of variables, domains, and constraints.

- **Global Constraint:** A sophisticated constraint (e.g., `alldifferent`) that encapsulates complex multi-variable relationships and utilizes highly optimized algorithms to reduce domains efficiently.
- **Auxiliary Variables:** Additional variables introduced into a model to simplify the mathematical expression of constraints and enable the use of global constraints.
- **Implied Constraints:** Constraints logically derived from the problem's rules that are explicitly added to the model to help the solver deduce dead-ends earlier.
- **Symmetry Breaking:** The technique of adding artificial ordering constraints to eliminate structurally equivalent solutions, preventing the solver from repeating identical search paths.
- **Channeling Constraints:** Logical equivalencies used to bind two different mathematical representations (Dual Viewpoints) of the same problem together, ensuring they update synchronously during the search.
- **Lexicographic Ordering ($lex \leq$):** A constraint ensuring one sequence of variables is sorted before or equal to another sequence, heavily used in breaking symmetries within matrix models.